

MICROJUEGO ASTEROIDS

- **Género: Shooter 2D**
- **Motor: Unity 6**
- **Autora: Berta Ruiz Lara**

1. Resumen

El juego gira en torno a una única escena (SampleScene) que contiene todo el gameplay: nave espacial del jugador, asteroides enemigos, fondo espacial e interfaz del usuario. El jugador debe eliminar los asteroides lanzándoles balas antes de que estos impacten contra él, resultando en un GameOver. Al reiniciar la partida, o al reanudarla tras haber pausado el juego anteriormente, se recarga la misma escena, restableciendo todas las variables de estado. La puntuación del jugador se va acumulando en el contador que aparece por pantalla denominado "SCORE".

2. Scripts

- **Player.cs:** Lleva a cabo el movimiento del jugador (rotación y velocidad) , el disparo, el efecto de pantalla infinita, la colisión con los asteroides y el reinicio de partida.
- **Asteroid.cs:** Controla el descenso de los asteroides, su autodestrucción tras un tiempo máximo de vida establecido, y un posible empuje lateral si este asteroide es un fragmento de otro asteroide que ha sido impactado por una bala del jugador.
- **Bullet.cs:** Controla el movimiento de la bala, detecta su colisión con los asteroides, reproduce un sonido al dispararse, y al impactar contra un asteroide. También actualiza la puntuación.
- **EnemySpawner.cs:** Calcula los límites para spawnear mediante la cámara, y genera oleadas de asteroides periódicamente.
- **Pause.cs:** Gestiona la pantalla y el estado de pausa, controlando la visibilidad de la pantalla correspondiente.
- **TextEffect.cs:** Gestiona el texto que indica al jugador la posibilidad de poder pausar la pantalla mediante la tecla Escape.

3. Mecánicas:

- **Movimiento:** la nave rota según el eje horizontal de entrada, y se impulsa hacia adelante aplicando una fuerza en la dirección `transform.right`
- **Disparo (+ sonido extra):** al pulsar el botón de salto (tecla Espacio por defecto), se instancia una bala en la posición de `BulletSpawner` y se reproduce un sonido. Si se impacta en un asteroide, también se reproduce un sonido tras su destrucción.
- **Spawn de enemigos:** `EnemySpawner` instancia asteroides. Se va aumentando la frecuencia de aparición para ir escalando la dificultad. La posición horizontal de cada asteroide es aleatoria dentro del ancho visible de la cámara.
- **Puntuación:** una variable que se incrementa cada vez que un asteroide (o fragmento) es destruido por el jugador. Se va actualizando en directo por pantalla.
- **Efecto pantalla infinita:** en lugar de paredes físicas, la nave calcula los límites visibles de la cámara en `Start()` y, si su posición supera esos límites por cualquiera de los 4 lados, reaparece automáticamente por el lado opuesto (`Update()`).
- **Fragmentación de enemigos (extra):** cada asteroide tiene un campo `canDivide`. Los asteroides originales lo tienen por defecto a `true`; al recibir el primer impacto, en lugar de destruirse sin más, generan dos fragmentos más pequeños (mitad de escala) con un pequeño impulso lateral aleatorio y `canDivide = false`. Estos fragmentos, al ser impactados de nuevo, son destruidos de manera definitiva. La puntuación que obtiene el jugador siendo asteroide primerizo, o un fragmento, es exactamente la misma.
- **Pausa/Reinicio (extra):** al pulsar la tecla `Escape`, `Time.timeScale` pasa a 0, congelando físicas y movimiento (pero no la lectura de `Input`), y se muestra un panel de pausa con un botón para reanudar. Al colisionar con un asteroide, se muestra un panel de `Game Over` y el juego queda congelado hasta que el jugador pulsa `Espacio`, momento en que se recarga la escena.

Como propuesta de mejora, creo podría haber hecho que al pausar el juego, el input dejase de leerse (a excepción de clicar en el botón de “RESUME”).

4. Interacción:

- **Avance y retroceso:** flechas arriba y abajo.
- **Giro:** flechas izquierda y derecha.
- **Disparo:** tecla espacio.
- **Pausa del juego:** tecla escape.
- **En pantalla Pausa:** pulsar con el ratón el botón “RESUME”.
- **En pantalla GameOver:** tecla espacio para reanudar el juego.